1.

The third element of an array has index 2.

2.

```
int[] quantities = new int[20];
```

3.

```
double[] heights = {1.65, 2.15, 4.95};
```

4.

```
for (int grade : grades) {
  System.out.println(grade);
```

5.

a) Insert algorithm:

1. Start at the end of the array.
2. Move each element one position to the right until reaching the insertion index.
3. Place the new value in the open position.

b) Delete algorithm:

1. Start at the deletion index.
2. Move each element one position to the left to fill the gap.
3. (Optional) Set the last element to a default value.

6.

Passing an entire array passes a reference to the array (so changes affect the original), while passing a single element passes only that value (a copy for primitives).

7.

Offset indexes are required because arrays start at index 0, while counting in real life often starts at 1.

8.

```
String name = "Elaine";
System.out.println(name.charAt(3));
```

Output: i

9. Array vs ArrayList

a) Accessing an element:

- Array: arr[i]
- ArrayList: list.get(i)

b) Adding an element:

- Array: Must manually resize or shift elements
- ArrayList: list.add(value)

c) Deleting an element:

- Array: Must manually shift elements
- ArrayList: list.remove(index)

d) Assigning a new value:

- Array: arr[i] = value
- ArrayList: list.set(i, value)

e) Determining size:

- Array: arr.length
- ArrayList: list.size()

10.
A dynamic array (ArrayList) is better when the number of elements is unknown or changes often, such as storing user input.

11.
The indexOf() method uses the .equals() method to check if objects are equal

12.

Wrapper class objects are compared using .equals() instead of ==

13.

a) True
b) True
c) True
d) False elements are initialized to 0, not 1
e) True
f) True
g) False a linear search may check every element
h) False $4 \times 4 = 16$ elements, not 8
i) False lowercase letters have higher Unicode values
j) False array size cannot change
k) True
l) False must use wrapper classes like Integer